

Notebook-Workflows und Modularisierung

Sobald Analysen komplexer werden, stellt sich die Frage: Wie ruft man Notebooks gegenseitig auf, übergibt Parameter und teilt Code wieder? Diese Seite stellt vier Methoden vor und erklärt, wann welche sinnvoll ist.

Die vier Methoden im Überblick

Methode	Haupteinsatzzweck
Lakeflow Jobs	Produktionsreife Orchestrierung mit Abhängigkeiten und Zeitplänen
<code>%run</code>	Ein anderes Notebook inline einbinden (Code-Modularisierung)
<code>dbutils.notebook.run()</code>	Notebooks mit Parametern und Rückgabewerten dynamisch aufrufen
Workspace-Dateien	Code in wiederverwendbaren Dateien organisieren (empfohlen für Modularisierung)

Lakeflow Jobs (empfohlen für Orchestrierung)

Der bevorzugte Ansatz für produktive Umgebungen. Ermöglicht komplexes Workflow-Management mit Abhängigkeiten zwischen Aufgaben (Tasks), Zeitplänen und ereignisbasierten Auslösern (Triggern). Erfordert im Gegenzug mehr Einrichtungsaufwand als die anderen Methoden.

`%run` – Notebook inline einbinden

`%run` führt ein anderes Notebook direkt innerhalb der aktuellen Ausführung aus; dessen Funktionen und Variablen stehen danach im aktuellen Notebook zur Verfügung.

- Muss eine ganze Zelle für sich allein belegen.
- Kann **keine Parameter** übergeben (außer über Widgets mit Standardwerten).
- Kann **keine Rückgabewerte** liefern.

- Widgets im aufgerufenen Notebook behalten ihre Standardwerte, sofern nicht explizit überschrieben.
- Kann keine reinen Python-Dateien direkt importieren – dafür sind Workspace-Dateien oder installierte Bibliotheken nötig.

dbutils.notebook.run() – dynamischer Aufruf mit Parametern

Startet einen kurzlebigen ("ephemeral") Job, um ein Notebook auszuführen. Signatur:

```
run(path, timeout_seconds, arguments)
```

- **Parameter übergeben:** `arguments` ist eine Schlüssel-Wert-Zuordnung, die Widget-Werte im Zielnotebook setzt.
- **Rückgabewerte:** Parameter und Rückgabewerte müssen Zeichenketten (Strings) sein.
- **Timeout:** `timeout_seconds = 0` bedeutet kein Timeout. Bei mehr als 10 Minuten Systemausfall schlägt der Aufruf unabhängig vom Timeout fehl.
- Jobs, die über diese API gestartet werden, müssen innerhalb von 30 Tagen abgeschlossen sein.
- Der `arguments`-Parameter akzeptiert nur ASCII-Zeichen (lateinisches Alphabet) – Sonderzeichen führen zu Fehlern.

Vergleich: `%run` bindet Code sofort ein, ohne separaten Job – einfach, aber ohne Parameter/Rückgabewerte. `dbutils.notebook.run()` erstellt für jeden Aufruf einen eigenen Job – mit mehr Overhead, dafür flexibler.

Workspace-Dateien (empfohlen für Modularisierung)

Code wird in wiederverwendbare Dateien im Workspace strukturiert, unterstützt Versionskontrolle über Repositories und IDE-Integration. Erleichtert Debugging und Unit-Testing im Vergleich zu Code, der direkt im Notebook verstreut ist.

Strukturierte Daten zwischen Notebooks übergeben

Drei gängige Strategien:

- **Temporäre Views:** Das aufgerufene Notebook erstellt eine globale temporäre View und gibt deren Namen zurück; das aufrufende Notebook liest daraus.

- **Speicherung auf DBFS:** Für größere Datenmengen wird das Ergebnis als Datei (z. B. Parquet) gespeichert und der Pfad zurückgegeben.
- **JSON-Serialisierung:** Mehrere Rückgabewerte werden mit `json.dumps()` (Python) bzw. entsprechenden Bibliotheken (Scala) codiert und im aufrufenden Notebook wieder decodiert.

Fehlerbehandlung

Bei Fehlern wirft die API eine `WorkflowException`. Übliche Try-Catch-Muster mit ansteigender Wiederholungslogik (Retry) werden für Python und Scala in der Dokumentation demonstriert.

Parallele Ausführung

Notebooks können auch gleichzeitig mittels Threads oder Futures (Scala und Python) ausgeführt werden, für parallele Verarbeitung unabhängiger Teilaufgaben.

Wichtige Einschränkungen

- `dbutils`-APIs stehen nur in Python und Scala zur Verfügung; `dbutils.notebook.run()` kann aber auch R-Notebooks aufrufen.
- Auf Serverless Compute wird empfohlen, DataFrame-APIs statt RDD-APIs zu verwenden (Kompatibilitätsgründe).
- Absolute Pfade folgen dem Muster `/Users/name@firma.com/ordner/notebook`, relative Pfade `./notebook-name`.

Empfehlung

`dbutils.notebook.run()` eignet sich besonders für Fälle, in denen Lakeflow Jobs nicht ausreichen – zum Beispiel metadatengetriebene ETL-Prozesse mit dynamisch wechselnden Notebook-Aufrufen. `%run` eignet sich gut zum schnellen Prototyping, führt aber zu eng gekoppeltem Code, der in Produktionsumgebungen schwerer zu warten ist.

Quelle: <https://docs.databricks.com/aws/en/notebooks/notebook-workflows>
Stand: 2026-08-06